

Диплом

ВВЕДЕНИЕ

Современные информационные системы всё чаще оперируют непрерывными потоками данных: журналами действий пользователей, телеметрией сервисов, событиями взаимодействия с внешними API. Объём таких потоков в крупных интернет-компаниях измеряется миллионами сообщений в секунду, а требования к задержке обработки сводятся к единицам секунд. Для построения подобных конвейеров широко применяется фреймворк Apache Spark и его подсистема структурированной потоковой обработки Spark Structured Streaming, обеспечивающая декларативное описание преобразований над неограниченными таблицами и масштабируемое исполнение на кластере.

Ключевым требованием к промышленным потоковым конвейерам является корректность данных в выходных хранилищах. На практике это означает гарантию строго однократной обработки (exactly-once): каждое входное сообщение должно быть отражено в результирующей таблице ровно один раз вне зависимости от перезапусков, сбоев исполнителей и иных нестандартных ситуаций. Нарушение данной гарантии ведёт к появлению дубликатов или, наоборот, к утере данных, что неприемлемо для финансовой отчётности, биллинга, систем антифрода и любых аналитических процессов, опирающихся на агрегаты.

В качестве целевого хранилища в настоящей работе рассматривается распределённая платформа YTsaurus. Платформа объединяет в себе свойства файловой системы, документной базы данных и системы массовой аналитической обработки, причём её реляционная подсистема — динамические таблицы — поддерживает ACID-транзакции с серьёзной изоляцией. Дополнительно YTsaurus предоставляет встроенный механизм очередей с потребителями, позволяющий описывать поток событий и его продвижение в едином формате с табличными данными. Интеграция YTsaurus с Apache Spark реализована в проекте SPYT, поэтому именно эта связка обеспечивает наиболее короткий путь от потокового источника к транзакционному хранилищу. Сочетание упомянутых свойств делает YTsaurus естественной целевой системой для исследования механизма распределённых транзакций в Spark Structured Streaming.

Текущая реализация потокового адаптера в SPYT обеспечивает только семантику не реже одного раза (at-least-once): данные записываются в динамическую таблицу одним вызовом, продвижение смещения потребителя выполняется отдельно после завершения записи, и при сбое между этими двумя действиями возникает повторная обработка батча. Такое поведение неприемлемо для прикладных задач, связанных с финансовыми и аналитическими расчётами, и требует разработки механизма, объединяющего запись данных и продвижение смещения в единую распределённую транзакцию.

Объектом исследования является потоковая обработка данных в распределённых вычислительных системах. Предметом исследования — механизм распределённых транзакций для динамических таблиц YTsaurus в адаптере SPYT, обеспечивающий гарантию строго однократной обработки потока в Apache Spark Structured Streaming.

Цель работы — разработать и реализовать механизм поддержки распределённых транзакций для динамических таблиц YTsaurus в адаптере SPYT, обеспечивающий exactly-once семантику записи потоковых данных.

Для достижения поставленной цели сформулированы следующие задачи:

1. Проанализировать архитектуру Apache Spark Structured Streaming и текущую реализацию потокового адаптера SPYT с точки зрения семантики доставки.
2. Изучить модель транзакций YTsaurus и особенности размещения грс-прокси в архитектуре SPYT, влияющие на возможность распределённой координации.
3. Спроектировать архитектуру модуля интеграции, обеспечивающего атомарную фиксацию пары «запись данных в динамическую таблицу — продвижение смещения потребителя».
4. Реализовать спроектированный механизм в исходном коде SPYT с использованием точек расширения Apache Spark.
5. Разработать набор тестов, подтверждающих корректность семантики строго однократной обработки в условиях штатной работы и при моделировании сбоев.
6. Оценить производительность полученного решения и сравнить рассмотренные архитектурные варианты.

Научная новизна работы заключается в впервые предложенной и реализованной схеме координации tablet-транзакций YTsaurus с микропакетным циклом исполнения Spark Structured Streaming. Практическая значимость состоит в том, что разработанный модуль может быть напрямую использован в производственных потоковых конвейерах, построенных на YTsaurus и SPYT, для перехода с гарантии at-least-once к exactly-once без изменения пользовательского кода.

Методы исследования включают анализ исходного кода Apache Spark и SPYT, проектирование архитектуры на основе паттернов внедрения зависимостей через ServiceLoader и инструментации байт-кода через Java-агента, а также экспериментальную оценку реализованного механизма с использованием локального развёртывания YTsaurus.

Работа состоит из введения, пяти глав, заключения и списка использованных источников. В первой главе анализируется проблема консистентности данных в потоковой обработке и обосновывается выбор YTsaurus как целевого хранилища. Во второй главе рассматривается модель транзакций YTsaurus и особенности её реализации в архитектуре SPYT. Третья глава посвящена проектированию модуля интеграции. В четвёртой главе описывается реализация. Пятая глава содержит описание методики тестирования и результаты экспериментов. Заключение подводит итоги работы и намечает направления дальнейших исследований.

1 АНАЛИЗ ПРОБЛЕМЫ КОНСИСТЕНТНОСТИ ДАННЫХ

В настоящей главе рассмотрены гарантии доставки в системах потоковой обработки, проанализирована архитектура Apache Spark Structured Streaming, описаны основные понятия платформы YTsaurus и обоснован её выбор в качестве целевого хранилища, изучена исходная реализация потокового адаптера SPYT и формализована решаемая проблема.

1.1 Гарантии доставки в потоковой обработке

1.1.1 Семантика at-most-once, at-least-once и exactly-once

В теории распределённых систем принято выделять три уровня гарантий доставки сообщений между источником и приёмником [1, 2]. Семантика не более одного раза (at-most-once) допускает потерю сообщений, но исключает появление дубликатов:

после отправки сообщения отправитель не запрашивает подтверждения, и при сбое сообщение теряется безвозвратно. Семантика не реже одного раза (at-least-once) гарантирует, что каждое сообщение будет в конечном итоге доставлено, однако допускает повторную доставку при сбоях. Семантика строго однократной обработки (exactly-once) сочетает гарантию доставки с гарантией единственности: каждое сообщение отражается в результате ровно один раз. Именно эта семантика необходима в задачах, где входной поток описывает события с финансовым или юридическим смыслом, либо вычисляются агрегаты, чувствительные к дубликатам.

Следует подчеркнуть, что строгая exactly-once семантика для произвольной комбинации источника и приёмника недостижима в принципе [3]. Достижимым на практике является вариант exactly-once в рамках конкретной выходной системы: каждое сообщение приводит к ровно одному эффекту в выходной таблице вне зависимости от того, сколько раз потоковый движок его обработает.

1.1.2 Существующие методы решения проблемы дубликатов

Для устранения дубликатов в потоковой обработке применяются три класса методов.

Решения на уровне приложения. Пользователь дополняет выходные строки уникальным ключом и реализует в приёмнике upsert-логику: повторная запись с тем же ключом не приводит к новому эффекту. Подход требует наличия естественного ключа в данных или возможности его сгенерировать (например, на основании смещения исходной очереди), а также поддержки upsert в целевом хранилище. К ограничениям относятся: невозможность применения к агрегатам и иным преобразованиям, при которых выходная строка не имеет однозначного соответствия с входной, а также накладные расходы на хранение и проверку ключей. Конкретная реализация этого подхода для платформы YTsaurus и её ограничения рассматриваются в § 1.5.

Пост-обработка дедупликации. В выходную таблицу пишутся все строки, после чего отдельный процесс выполняет периодическую дедупликацию. Подход прост в реализации, но не решает проблему в реальном времени, удваивает объём ввода-вывода и требует наличия признака, по которому возможна дедупликация.

Транзакционная фиксация. Запись данных и фиксация прогресса чтения объединяются в одну атомарную операцию в выходной системе, что исключает рассогласование между состоянием выходной таблицы и состоянием курсора потребителя [4]. Подход не требует изменения пользовательской модели данных и не вносит дополнительных задержек на обработку, однако предъявляет требования к выходной системе: она должна поддерживать транзакции, объединяющие модификацию данных и продвижение курсора.

В настоящей работе используется третий подход. Его выбор обусловлен тем, что динамические таблицы YTsaurus поддерживают полноценные ACID-транзакции, объединяющие операции записи и операции продвижения смещения потребителя очереди в одной транзакции, и не требуется вводить искусственные ключи в пользовательскую модель данных.

1.2 Архитектура Apache Spark Structured Streaming

1.2.1 Микропакетная модель исполнения

Apache Spark Structured Streaming реализует обработку неограниченного потока событий в виде последовательности микропакетов [5]. Поток рассматривается как непрерывно растущая таблица, к которой пользователь применяет реляционные преобразования. Движок периодически отрезает от потока новый фрагмент данных, выполняет над ним пользовательский план запроса и записывает результат в целевую систему. Каждый такой фрагмент имеет монотонно возрастающий идентификатор батча.

Контроль за продвижением потока возложен на класс `MicroBatchExecution`, реализующий цикл исполнения. На каждой итерации цикл выполняет следующие шаги:

1. Опрос источника на предмет нового максимального смещения.
2. Запись смещения батча в журнал смещений (offset log).
3. Вычисление логического плана для интервала смещений и его материализация на исполнителях (метод `runBatch`).
4. Передача результата приёмнику через `Sink.addBatch`.
5. Запись идентификатора завершённого батча в журнал коммитов (commit log).
6. Уведомление источников о коммите через `Source.commit`.

1.2.2 Точки расширения для источников и приёмников

Реализация конкретного источника обязана наследоваться от трейта `Source` и предоставить методы получения текущего максимального смещения, формирования датафрейма для интервала смещений и обработки уведомления о коммите. Реализация приёмника наследуется от трейта `Sink` и предоставляет единственный метод `addBatch(batchId, data)`, в котором приёмник обязан гарантировать запись данных батча с заданным идентификатором. Соглашение Spark предусматривает, что приёмник самостоятельно отвечает за идемпотентность записи в случае повторного вызова `addBatch` с тем же идентификатором, но не диктует механизм этой идемпотентности.

1.2.3 Управление смещениями и контрольные точки

Журналы смещений и коммитов хранятся в каталоге контрольной точки, путь к которому задаётся пользователем. Содержимое журналов представляет собой пронумерованные файлы, каждый из которых описывает соответственно начатый или завершённый батч. Источник несёт ответственность за сериализацию и десериализацию своего смещения. После перезапуска движок Spark восстанавливает состояние из журналов: батч считается завершённым, если в журнале коммитов присутствует запись с соответствующим идентификатором; в противном случае батч переисполняется с теми же границами смещений.

Указанный механизм гарантирует семантику *at-least-once* на уровне Spark в предположении, что приёмник не теряет данных. Достижение *exactly-once* требует дополнительной координации между приёмником и источником, выходящей за рамки штатных интерфейсов Spark.

1.3 Платформа YTsaurus

1.3.1 Общая архитектура и виды таблиц

YTsaurus — распределённая платформа хранения и обработки данных, развиваемая компанией Яндекс и открытая под лицензией Apache 2.0 [6]. Платформа предоставляет иерархическое пространство имён `Supress`, в узлах которого размещаются файлы, документы, а также два класса таблиц: статические и динамические. Статические таблицы предназначены для пакетной аналитической обработки, поддерживают форматы строчного и колоночного хранения и оптимизированы для последовательного чтения большими блоками. Динамические таблицы являются распределённой подсистемой структурированных таблиц YTsaurus, поддерживают точечный доступ по ключу, строгую сортировку и шардирование по диапазонам, обеспечивают строгую согласованность и сериализуемую изоляцию транзакций.

1.3.2 Очереди и потребители

Поверх динамических таблиц YTsaurus предоставляет высокоуровневую абстракцию очередей и потребителей [6]. Очередь — это упорядоченная динамическая таблица, в которую события добавляются в конец и читаются в порядке добавления. Внутри очередь разбита на таблетки, каждый из которых имеет собственный монотонно возрастающий индекс строки. Потребитель описывается отдельной динамической таблицей, в строках которой для каждой пары «очередь — индекс таблетки» хранится текущее смещение чтения. Продвижение смещения выполняется специальной операцией `advanceConsumer`, доступной как в виде самостоятельного запроса, так и в составе транзакции.

Сочетание абстракций «очередь» и «потребитель» в едином хранилище принципиально важно для настоящей работы: и пользовательские табличные данные, и состояние курсора чтения находятся в одной системе, поддерживающей транзакции, что открывает возможность их атомарной модификации.

1.3.3 Обоснование выбора YTsaurus как целевой системы хранения

Выбор YTsaurus в качестве целевого хранилища обусловлен несколькими свойствами, существенными именно для задачи строго однократной потоковой обработки. Во-первых, динамические таблицы YTsaurus поддерживают полноценные ACID-транзакции, в одной транзакции допускаются как операции изменения строк (`modifyRows`), так и операции продвижения смещения потребителя (`advanceConsumer`). Это позволяет атомарно фиксировать пару «запись результата батча — продвижение курсора чтения», что является ключевым условием `exactly-once`. Альтернативные хранилища, лишённые транзакций над курсором потребителя, требуют введения внешнего механизма координации, как правило, с использованием искусственных идемпотентных ключей.

Во-вторых, абстракции очереди и потребителя в YTsaurus реализованы поверх той же динамической таблицы, что и пользовательские данные, и используют единый механизм репликации и согласованности. Это исключает межсистемную координацию между брокером сообщений и хранилищем, типичную для связки Apache Kafka и внешнего хранилища.

В-третьих, для платформы YTsaurus уже существует адаптер SPYT, обеспечивающий глубокую интеграцию с Apache Spark: поддержку чтения и записи статических и динамических таблиц, передачу схемы, оптимизатор сортированных таблиц и базовый потоковый адаптер. Использование SPYT в качестве основы позволяет сосредоточиться на собственно проблеме распределённых транзакций, не тратя усилий на разработку соединителя с нуля.

В-четвёртых, в инфраструктуре Яндекса YTsaurus является основной платформой хранения данных, и потоковые конвейеры, доставляющие данные в динамические таблицы YTsaurus, представляют собой массовый прикладной сценарий.

1.4 Адаптер SPYT и исходная реализация потокового модуля

В настоящем разделе описывается исходное состояние потокового модуля SPYT, соответствующее ветке `branch-2.7.x` репозитория проекта, до начала работ, выполненных в рамках настоящей ВКР.

1.4.1 Состав модуля потоковой обработки

Поддержка потоковой обработки в SPYT реализована в модуле `data-source` и представлена следующими компонентами:

- `YtStreamingSource` — реализация трейта `Source`, читающая данные из очереди `YTsaurus` от имени заданного потребителя.
- `YtStreamingSink` — реализация трейта `Sink`, выполняющая запись в динамическую таблицу `YTsaurus`.
- `YtQueueOffset` — представление смещения батча, содержащее карту «индекс таблета — последний прочитанный индекс строки».
- `YtQueueRange` — Spark-партиция, описывающая полуоткрытый интервал индексов строк в одном таблете.
- `YtQueueRDD` — `RDD`, реализующий чтение интервала из таблета через операцию `pullConsumer`.

1.4.2 Поток данных

В методе `YtStreamingSource.latestOffset` запрашивается атрибут `queue_partitions` очереди и сравнивается с последним подтверждённым смещением, прочитанным из таблицы потребителя. На основании разности формируется новое максимальное смещение батча. В методе `YtStreamingSource.getBatch` разность смещений преобразуется в набор объектов `YtQueueRange` и оборачивается в `YtQueueRDD`. Каждая партиция `RDD` соответствует ровно одному таблету очереди, в чём состоит свойство 1:1 соответствия партиций источника и планшетов очереди.

В методе `YtStreamingSink.addBatch` полученный датафрейм проходит через `foreachPartition`, в каждой партиции создаётся локальный экземпляр `YtDynamicTableWriter` и выполняется построчная запись через операцию `modifyRows` без какой-либо транзакции — каждый запрос автокоммитится клиентской библиотекой YT по факту отправки. После завершения `addBatch` Spark вызывает `YtStreamingSource.commit`, в котором единственный метод `YtQueueOffset.advance` создаёт собственную короткоживущую `tablet`-транзакцию, выполняет в ней `advanceConsumer` для каждого таблета и фиксирует её. Эта транзакция охватывает только продвижение смещения и не связана с записью данных, выполненной выше.

1.4.3 Семантика исходной реализации

В исходной реализации запись данных и продвижение смещения выполняются в двух полностью независимых контекстах: запись — на исполнителях через `foreachPartition` без какой-либо транзакции, продвижение — на драйвере в собственной короткоживущей `tablet`-транзакции, охватывающей исключительно `advanceConsumer`. Между завершением записи и завершением продвижения существует временной интервал, на котором система может выйти из строя, и атомарность пары «запись результата батча — фиксация прогресса чтения» не обеспечивается. Дополнительно, отсутствие транзакции вокруг записи означает, что частично завершённый батч оставляет в выходной таблице произвольное подмножество строк без возможности отката.

1.4.4 Сценарии нарушения консистентности

Рассмотрим сценарии сбоев, в каждом из которых исходная реализация SPYT не обеспечивает `exactly-once`.

Сценарий А (сбой между `addBatch` и `commit`). Запись в динамическую таблицу успешно завершена, идентификатор батча зафиксирован в журнале коммитов Spark, но процесс драйвера падает до того, как `Source.commit` успел продвинуть смещение потребителя. После перезапуска Spark считает батч завершённым и не переисполняет его, однако смещение потребителя по-прежнему указывает на начало батча. Возникает рассогласование между состоянием выходной таблицы и состоянием курсора потребителя; при последующих перезапусках или удалении контрольной точки рассогласование приводит к появлению дубликатов в выходной таблице.

Сценарий Б (сбой во время `addBatch`). Часть партиций успела выполнить запись, часть — нет. Идентификатор батча в журнал коммитов не записан, поэтому Spark переисполнит батч. Данные, успевшие попасть в динамическую таблицу при первой попытке, окажутся в ней дважды.

Оба сценария устраняются объединением записи данных и продвижения смещения в одну распределённую транзакцию: при сбое транзакция отменяется целиком, а успешная фиксация одновременно делает наблюдаемыми и записанные данные, и новое смещение потребителя. Общая причина возникновения сценариев — запись данных и продвижение курсора потребителя не образуют единого атомарного действия.

1.5 Существующее частичное решение *exactly-once* в SPYT: запись в сортированные таблицы

До начала работ, выполненных в рамках настоящей ВКР, в SPYT существовал способ получения семантики *exactly-once* без транзакционной координации записи и продвижения смещения. Способ основан на идемпотентной записи в сортированные динамические таблицы `YTsaurus` и был ранее реализован автором настоящей работы.

1.5.1 Архитектура решения

Целевая таблица создаётся как сортированная динамическая таблица, первичным ключом которой служит триплет «путь к очереди-источнику, индекс таблета очереди, индекс строки в таблете». Поля ключа автоматически проставляются приёмником в каждой выходной строке исходя из её происхождения во входной очереди: значения индексов таблета и строки переносятся из метаданных операции `pullConsumer` без участия пользователя. Запись в таблицу осуществляется операцией `upsert`: повторное появление строки с тем же ключом перезаписывает существующую строку без накопления дубликатов. Продвижение смещения потребителя выполняется в отдельной транзакции после успешной записи, как и в исходной *at-least-once*-реализации, но семантическая корректность не нарушается: даже если повторно исполненный батч повторно запишет уже зафиксированные строки, их ключи совпадут с уже хранящимися и `upsert` не создаст дубликатов.

1.5.2 Ограничения существующего решения

Описанное решение не покрывает класс задач, для которого формулируется настоящая работа, в силу двух существенных ограничений.

Первое ограничение — производительность. Сортированная динамическая таблица `YTsaurus` поддерживает первичный ключ, по которому строки физически упорядочены, что приводит к дополнительным накладным расходам по сравнению с упорядоченной таблицей: на каждую вставку выполняется поиск точки вставки в B-дереве, при достижении компактификационных порогов запускается фоновая компактификация диапазонов, а размер хранимого индекса растёт пропорционально объёму данных. На малых объёмах разница незаметна, однако на больших таблицах деградация записи становится существенной и продолжает расти с ростом объёма таблицы. Упорядоченная таблица, не имеющая первичного ключа, лишена этих накладных расходов и обеспечивает существенно более высокую пропускную способность записи.

Второе ограничение — несовместимость с произвольными преобразованиями. Идемпотентный ключ строится из координат входной строки в очереди и предполагает однозначное соответствие между входной и выходной строкой. При преобразованиях, нарушающих это соответствие — таких как соединения с другими источниками, оконные функции, агрегации, дедупликация по доменным признакам — выходная строка не имеет уникальных координат во входной очереди, и уникальность ключа невозможно обеспечить. В таких сценариях идемпотентная

запись через сортированную таблицу принципиально неприменима, и единственным способом сохранить exactly-once остаётся транзакционная координация записи и продвижения смещения.

Из перечисленных ограничений следует, что существующее частичное решение покрывает только узкий класс задач с преобразованиями без сохранения состояния и малым объёмом данных и не может быть рекомендовано как общее средство получения exactly-once в SPYT. Возникает потребность в механизме, который обеспечивает exactly-once для произвольных преобразований и для записи в упорядоченные таблицы YTsaugus, не уступая существующему решению по производительности.

1.6 Постановка задачи

Задача, решаемая в настоящей работе, формулируется в полном соответствии с техническим заданием.

Цель работы: обеспечение гарантии строго однократной потоковой обработки данных в Apache Spark при записи в динамические таблицы YTsaugus.

Содержание задачи. Требуется разработать механизм, который атомарно фиксирует два действия в рамках обработки одного микропакета:

1. запись результата обработки порции данных в целевую динамическую таблицу YTsaugus;
2. сохранение прогресса чтения в систему контрольных точек Apache Spark и YTsaugus.

В случае сбоя на любом этапе обработки порции данных распределённая транзакция должна полностью откатываться, исключая появление дубликатов или потерю данных. После фиксации транзакции состояние выходной таблицы и состояние курсора потребителя должны быть согласованы между собой и с журналом коммитов Spark.

Требования к решению. Разрабатываемый механизм должен:

- обеспечивать exactly-once при записи в упорядоченные динамические таблицы YTsaugus;
- сохранять корректность при произвольных преобразованиях пользовательского запроса, включая соединения, оконные функции и агрегации;
- не уступать существующему частичному решению по пропускной способности записи на задачах, где последнее применимо;
- использовать штатные механизмы транзакций YTsaugus и существующие в SPYT средства расширения Apache Spark — адаптеры и инструментацию байт-кода через Java-агент;
- не требовать изменений в пользовательском коде потокового запроса; переключение между исходной семантикой at-least-once и новой семантикой exactly-once должно выполняться установкой одного конфигурационного параметра.

2 МОДЕЛЬ ТРАНЗАКЦИЙ YTSAURUS

В настоящей главе рассмотрена транзакционная модель платформы YTsaugus: типы транзакций, их свойства и API клиентской библиотеки, а также особенности размещения грс-прокси в архитектуре SPYT, существенно влияющие на возможность реализации распределённой транзакции.

2.1 Типы транзакций YTsaurus

YTsaurus поддерживает два самостоятельных типа транзакций, различающихся областью действия и механизмом реализации [6].

2.1.1 Master-транзакции

Master-транзакции реализуются распределённой подсистемой Master-серверов, обслуживающей метаданные Cypress. Состояние master-транзакции реплицируется между серверами и переживает сбои отдельных узлов. В master-транзакциях допускаются операции над файлами, документами и статическими таблицами Cypress, операции изменения структуры пространства имён, а также чтение динамических таблиц. Запись в динамические таблицы в master-транзакции не поддерживается, поскольку запись адресуется к подсистеме таблет-узлов, не участвующих в данном протоколе.

2.1.2 Tablet-транзакции

Tablet-транзакции реализуются подсистемой таблет-узлов и предназначены для работы с динамическими таблицами и очередями. В одной tablet-транзакции допускается выполнение операций `modifyRows`, `lookupRows`, `selectRows`, `advanceConsumer` над произвольным числом динамических таблиц, расположенных на разных таблет-узлах. Атомарность фиксации обеспечивается двухфазным протоколом коммита, согласованным с метаданными Cypress.

Именно tablet-транзакции являются единственным механизмом, в котором может быть атомарно объединена запись пользовательских данных в выходную динамическую таблицу и продвижение смещения потребителя очереди.

2.2 Свойства tablet-транзакций

2.2.1 ACID-гарантии

Tablet-транзакции YTsaurus обеспечивают полный набор ACID-свойств. Атомарность гарантируется двухфазным коммитом, согласованность поддерживается на уровне схем и ограничений целостности динамических таблиц, изоляция реализуется по схеме `serializable snapshot isolation`, надёжность обеспечивается репликацией `write-ahead log` на таблет-узлах [6].

2.2.2 Поддерживаемые операции

В контексте настоящей работы практически используются две операции:

- `modifyRows` — пакетная вставка, обновление и удаление строк динамической таблицы. Используется при записи результата батча в выходную таблицу.
- `advanceConsumer` — атомарное обновление смещения потребителя. Используется при фиксации завершённого батча.

Обе операции принимают идентификатор транзакции в качестве параметра либо вызываются на объекте транзакции, что обеспечивает их выполнение в рамках одного атомарного действия. Соответствующие обёртки над YT-клиентом представлены в модуле `yt-wrapper` адаптера SPYT в файлах `YtDynTableUtils` и `YtQueueUtils`.

2.3 Sticky-транзакции и роль грс-прокси

2.3.1 Размещение состояния транзакции

Принципиальной особенностью реализации tablet-транзакций в YTsaurus является то, что состояние транзакции (список изменений до коммита, идентификатор, таймстемпы) хранится не на таблет-узлах, а в клиентской библиотеке, исполняющейся внутри грс-прокси, через который клиент общается с кластером [6]. Таблет-узлы видят транзакцию только в момент её коммита, когда грс-прокси отправляет им сериализованный набор изменений. Такой выбор архитектуры существенно снижает нагрузку на таблет-узлы за счёт переноса буферизации мутаций на сторону прокси.

2.3.2 Параметр sticky

Поскольку состояние транзакции локально для одного грс-прокси, обращения к одной и той же транзакции должны направляться в один и тот же экземпляр прокси. Для этого в YTsaurus введён режим sticky: при создании транзакции с флагом `sticky=true` клиентская библиотека закрепляет за транзакцией конкретный грс-прокси и направляет в него все последующие запросы. В отсутствие данного флага запросы могут балансироваться между разными прокси, и любая операция над транзакцией с высокой вероятностью завершится ошибкой `No such transaction`.

В адаптере SPYT соответствующий код находится в `YtTransactionUtils.createTransaction`: тип транзакции выбирается равным `TransactionType.Tablet` при `sticky=true` и `TransactionType.Master` иначе.

2.3.3 Поддержание времени жизни транзакции

Tablet-транзакция имеет конечное время жизни, заданное параметром `transactionTimeout`. Чтобы транзакция не была автоматически прервана грс-прокси, клиент обязан периодически отправлять `ping`-запросы, продлевающие срок жизни. В клиентской библиотеке за это отвечает планировщик, активируемый параметрами `setPing(true)` и `setPingPeriod(...)` при создании транзакции. В коде SPYT данная конфигурация задаётся в `YtTransactionUtils.createTransaction`, а функция `pingTransaction` выполняет дополнительные синхронные `ping`-запросы в фоновом потоке.

2.4 Архитектура грс-прокси в SPYT

Стандартная схема развёртывания YTsaurus предполагает выделенный пул грс-прокси, общий для всех клиентов кластера. В этой схеме два процесса клиента, обращающиеся к кластеру, могут как попасть на один и тот же прокси, так и оказаться обслуживаемыми разными прокси, в зависимости от балансировщика.

Адаптер SPYT работает в специальной среде исполнения Apache Spark поверх YTsaurus, и в этой среде существует две конфигурации размещения грс-прокси.

2.4.1 Конфигурация с грс-прокси в job-proxy

В типичной для SPYT конфигурации грс-прокси встраивается непосредственно в `job-proxy` — процесс-обёртку, в котором YTsaurus запускает каждый компонент Spark (драйвер и каждый исполнитель). Такая схема обеспечивает локальность сетевых обращений и снижает нагрузку на общий пул прокси кластера. Однако следствием встраивания является то, что у каждого процесса Spark — собственный изолированный экземпляр грс-прокси, и состояние tablet-транзакции, созданной в драйвере, физически не доступно из исполнителей.

Поведение управляется конфигурационным параметром `spark.ytsaurus.rpc.job.proxy.enabled`, по умолчанию равным `true`. При значении `false` все компоненты Spark обращаются к общему пулу грс-прокси кластера.

2.4.2 Изоляция транзакций между процессами

В конфигурации с грс-прокси в `job-proxy` попытка передать идентификатор `tablet-транзакции`, созданной в драйвере, на исполнитель приводит к ошибке `No such transaction`. Эта ошибка указывает на то, что транзакция существует в одном грс-прокси, тогда как обращение к ней пришло в другой грс-прокси, который о её существовании не знает.

Изоляция между процессами устраняется одним из двух способов. Первый — сведение всех обращений к транзакции в один процесс Spark: тогда вся работа с транзакцией происходит через единственный грс-прокси, встроенный в `job-proxy` этого процесса, и проблема видимости не возникает. Второй — переход на общий пул грс-прокси (`spark.ytsaurus.rpc.job.proxy.enabled=false`) при условии, что транзакция создана в режиме `sticky` и клиентская библиотека на исполнителе настроена на использование того же прокси, что и драйвер; это требует расширения клиентской библиотеки YT функцией присоединения к существующей транзакции. Оба способа подробно рассматриваются в главе 3.

2.5 API клиентской библиотеки

2.5.1 Класс `ApiServiceTransaction`

Основным объектом, представляющим `tablet-транзакцию` в Java- и Scala-коде, является `ApiServiceTransaction`. Объект инкапсулирует идентификатор транзакции, ссылку на грс-прокси и предоставляет методы `modifyRows`, `advanceConsumer`, `commit`, `abort`, `ping`. Идентификатор транзакции доступен через метод `getId` и пригоден для логирования и передачи в качестве параметра в операции, поддерживающие `transactionId`.

2.5.2 Утилиты SPYT

В модуле `yt-wrapper` адаптера SPYT агрегированы обёртки над клиентской библиотекой:

- `YtTransactionUtils` — создание, коммит, отмена и пинг транзакций.
- `YtDynTableUtils` — операции над динамическими таблицами, включая `insertRows` с опциональным идентификатором родительской транзакции.
- `YtQueueUtils` — операции над очередями и потребителями. Содержит две перегрузки `advanceConsumer`: принимающую объект `ApiServiceTransaction` (вызов выполняется на самом объекте транзакции) и принимающую идентификатор родительской транзакции в виде строки (вызов выполняется на клиенте с указанием `transactionId` в запросе).

Наличие двух перегрузок отражает фундаментальное различие между сценарием «транзакция создана локально, объект доступен» и сценарием «транзакция создана на другом узле, локально известен только идентификатор». Первый сценарий пригоден для одного процесса, второй — для распределённой координации.

2.6 Ограничения и следствия для распределённой записи

Сведём наблюдения главы в следующие выводы.

1. Запись в выходную динамическую таблицу YTsaurus и продвижение смещения потребителя могут быть атомарно объединены только в одной tablet-транзакции.
2. Состояние tablet-транзакции локально для конкретного грс-прокси и не реплицируется между прокси.
3. В стандартной для SPYT конфигурации с грс-прокси, встроенным в job-проху, разные процессы Spark обслуживаются разными грс-прокси, и совместная работа с одной tablet-транзакцией возможна одним из двух способов: сведением всех обращений к транзакции в один процесс Spark либо переходом на общий пул грс-прокси с дополнительной поддержкой клиентской библиотеки — операцией присоединения к транзакции по её идентификатору и адресу прокси.

Перечисленные ограничения формируют пространство допустимых архитектурных решений, которое исследуется в главе 3.

3 ПРОЕКТИРОВАНИЕ МОДУЛЯ ИНТЕГРАЦИИ

В настоящей главе формулируются требования к модулю распределённой транзакции, описываются используемые точки расширения Apache Spark и SPYT, анализируется соответствие партиций потокового источника и приёмника, рассматриваются два архитектурных решения и обосновывается выбор одного из них.

3.1 Требования к модулю

3.1.1 Функциональные требования

К модулю предъявляются следующие функциональные требования.

1. Гарантия строго однократной обработки потоковых данных при записи из очереди YTsaurus в динамическую таблицу YTsaurus через Spark Structured Streaming.
2. Атомарность пары «запись батча в выходную таблицу — продвижение смещения потребителя на конец батча» в рамках единой tablet-транзакции YTsaurus.
3. Корректное поведение при сбоях: либо обе операции зафиксированы, либо обе откаты, и переисполнение батча не приводит к появлению дубликатов и потере данных.
4. Прозрачность для пользовательского кода: переключение между семантиками at-least-once и exactly-once выполняется конфигурационным параметром `spark.yt.streaming.transactional`, без модификации описания потокового запроса.

3.1.2 Нефункциональные требования

1. Совместимость с Apache Spark 3.5.x как с приоритетной поддерживаемой версией адаптера SPYT, при сохранении совместимости с Apache Spark 3.2.x.
2. Минимизация накладных расходов на батч в установившемся режиме работы.
3. Использование существующих в SPYT механизмов расширения Apache Spark — адаптеров и инструментации байт-кода через Java-агент.
4. Совместимость с существующими реализациями YtStreamingSource и YtStreamingSink: отсутствие необходимости перепроектирования базовых компонентов.

3.2 Точки расширения Apache Spark и SPYT

3.2.1 ServiceLoader и SPI

Часть точек расширения, доступных в API модуля интеграции, реализуется средствами стандартного механизма Java ServiceLoader. Реализация интерфейса регистрируется в файле META-INF/services/, и Spark или SPYT загружает её в момент инициализации. В реализации одного из решений, рассматриваемых в §3.4, используется интерфейс StreamingTransactionSupport, объявляющий контракт жизненного цикла распределённой транзакции, и его конкретная реализация YtsaurusStreamingTransactionSupport, регистрируемая через ServiceLoader.

3.2.2 Декорирование внутренних методов Spark

Часть необходимой функциональности располагается во внутренних классах Spark, не предусматривающих штатного расширения. Для воздействия на эти классы в SPYT применяется механизм инструментации байт-кода, реализованный в модуле spark-patch. Главный компонент модуля — Java-агент SparkPatchAgent, регистрируемый через параметр -javaagent JVM драйвера и исполнителей. Агент сканирует classpath в поиске классов-декораторов, помеченных аннотациями @PatchSource и @Decorate, и подменяет реализации соответствующих методов в исходных классах Spark, сохраняя оригинальное поведение и оборачивая его в пользовательский код.

В реализации одного из решений, рассматриваемых в §3.4, через данный механизм декорируется метод runBatch класса MicroBatchExecution с целью внедрения логики управления контекстом батча между вызовами Source.getBatch, Sink.addBatch и Source.commit.

3.2.3 ThreadLocal-контекст

Третья точка взаимодействия — ThreadLocal-переменные, используемые для передачи данных между методами одного потока без изменения сигнатур. В реализации одного из решений, рассматриваемых в §3.4, используется контекст YtStreamingTransactionContext, хранящий идентификатор родительской транзакции и адрес grpc-прокси, на которой была создана транзакция. Использование ThreadLocal оправдано тем, что обработка одного батча выполняется на драйвере в одном потоке, и контракт MicroBatchExecution гарантирует последовательное исполнение.

3.3 Соответствие партиций источника и приёмника

3.3.1 Соответствие 1:1 на уровне источника

В реализации источника соответствие между партициями RDD и таблетами очереди установлено явно. В классе YtQueueRange индекс Spark-партиции переопределён равным индексу таблета:

```
case class YtQueueRange(tabletIndex: Int, lowerIndex: Long,
                        upperIndex: Long) extends Partition {
  override def index: Int = tabletIndex
}
```

В методе YtQueueRDD.getPartitions партиции возвращаются отсортированными по tabletIndex, что обеспечивает стабильный порядок и взаимно однозначное соответствие между партициями RDD и таблетами очереди.

Свойство 1:1 соответствия позволяет в принципе организовать на каждом исполнителе независимую tablet-транзакцию, обрабатывающую один таблет: запись в выходную таблицу строк одного таблета и продвижение смещения потребителя для этого же таблета.

3.3.2 Потеря соответствия после преобразований

Однако данное свойство справедливо лишь до тех пор, пока RDD источника проходит через узкие преобразования (`map`, `filter`, `mapPartitions`), сохраняющие партиционирование. Любое широкое преобразование пользовательского пайплайна — `groupBy`, `join`, `repartition`, оконная агрегация — выполняет `shuffle` и пересоздаёт партиционирование результата. После `shuffle` партиция приёмника содержит строки, происходящие из произвольного подмножества таблеток источника, и независимое продвижение смещения по таблету становится невозможным: исполнитель не знает, какие таблетки внесли вклад в его партицию и в каком объёме.

Между источником и приёмником ничто не гарантирует отсутствие широких преобразований, поэтому решение, рассчитанное на сохранение партиционирования, нельзя считать применимым в общем случае.

3.3.3 Следствия для архитектуры

Из анализа следует, что распределённая координация транзакции должна допускать одно из двух:

- сведение всего батча к одному исполнителю, способному провести транзакцию целиком;
- предоставление множеству исполнителей возможности участвовать в одной общей транзакции, координируемой драйвером.

Эти два способа порождают два архитектурных варианта, рассмотренных далее.

3.4 Архитектурные решения

В §3.3.3 показано, что распределённая координация транзакции допускает либо сведение всего батча к одному исполнителю, либо вовлечение множества исполнителей в одну общую транзакцию, координируемую драйвером. Этим двум стратегиям соответствуют два архитектурных решения, рассмотренных ниже.

3.4.1 Решение 1: запись на одном исполнителе

Идея решения состоит в том, чтобы перед записью применять к датафрейму операцию `coalesce(1)`. После её применения весь батч обрабатывается одним заданием Spark на одном исполнителе. Этот исполнитель самостоятельно открывает tablet-транзакцию через локальный grpc-прокси, встроенный в его `job-proxy`, выполняет в этой транзакции запись данных батча и операцию `advanceConsumer` для всех таблеток очереди, после чего фиксирует транзакцию. На драйвере метод `Source.commit` при включённом флаге `spark.yt.streaming.transactional` пропускает действия по продвижению смещения, поскольку оно уже выполнено в составе фиксированной транзакции.

Жизненный цикл батча в решении 1 выглядит следующим образом.

1. На драйвере метод `Source.getBatch` формирует RDD из таблеток очереди.
2. На драйвере метод `Sink.addBatch` применяет к датафрейму `coalesce(1)` и запускает задание `foreachPartition` с одной партицией.
3. На выбранном исполнителе открывается tablet-транзакция через локальный grpc-прокси.

4. В этой транзакции выполняется построчная запись данных батча через `modifyRows`.
5. В этой же транзакции для каждого таблета очереди, попавшего в батч, выполняется операция `advanceConsumer` с соответствующим конечным смещением.
6. Транзакция фиксируется методом `commit`. При сбое на любом из шагов 4–6 транзакция отменяется методом `abort`, и Spark переисполняет батч.
7. На драйвере метод `Source.commit` пропускает действия по продвижению смещения.

С точки зрения реализации решение 1 локально для модуля `data-source-extended`: достаточно ввести альтернативную ветку в методе `YtStreamingSink.addBatch`, обрабатывающую при включённом флаге `spark.yt.streaming.transactional`, и зеркальную ветку в `YtStreamingSource.commit`, отключающую штатное продвижение смещения. Транзакция рождается, живёт и умирает внутри замыкания, переданного в `foreachPartition`, и не пересекает границу JVM. По этой причине решение 1 не требует ни инструментации байт-кода `MicroBatchExecution`, ни введения интерфейса `StreamingTransactionSupport`, ни `ThreadLocal`-контекстов передачи идентификатора транзакции между фазами обработки батча.

Сильная сторона решения — простота реализации, локализация изменений в одном модуле SPYT и отсутствие изменений в сторонних проектах. Решение опирается на штатный механизм `tablet-транзакций`, описанный в главе 2, и не требует расширения клиентской библиотеки YT.

Слабая сторона решения — потеря параллелизма записи. Операция `coalesce(1)` сводит все партии к одной без `shuffle`, и после её применения исходный план превращается в одну стадию из одного таска: пользовательские преобразования, чтение из таблетов очереди, формирование выходных строк, сериализация в формат `modifyRows` и сетевая передача в `гpc-прокси` выполняются в одном потоке одного процесса. На пути к динамической таблице возникают два узких места: пропускная способность одного исполнителя и пропускная способность одного `гpc-прокси`. Совместный потолок определяется минимумом двух, и обычно лимитирует именно исполнитель: данные не успевают сформироваться в темпе, при котором `прокси` становится узким местом. Дополнительно, накопление всего батча в памяти одного процесса повышает риск его исчерпания на больших батчах.

3.4.2 Решение 2: общий пул `гpc-прокси` и `AttachTransaction`

Идея решения состоит в том, чтобы изменить конфигурацию SPYT на общий пул `гpc-прокси` (флагом `spark.ytsaurus.rpc.job.proxy.enabled=false`) и расширить клиентскую библиотеку YT операцией присоединения исполнителя к существующей `tablet-транзакции`, открытой драйвером. На стороне библиотеки реализуется функция `attachTransaction(transactionId, proxyAddress)`, создающая локальный объект `ApiServiceTransaction`, который направляет запросы в указанный `гpc-прокси` с указанным идентификатором. Поскольку все исполнители работают с тем же `гpc-прокси`, что и драйвер, состояние `tablet-транзакции` остаётся видимым каждому из них (см. §2.4).

Жизненный цикл батча в решении 2 выглядит следующим образом.

1. На драйвере метод `Source.getBatch` формирует RDD из таблетов очереди.
2. На драйвере непосредственно перед `Sink.addBatch` открывается `tablet-транзакция`; её идентификатор и адрес `гpc-прокси`, на которой она открыта, сохраняются в `YtStreamingTransactionContext`.
3. Spark материализует RDD согласно пользовательскому плану и запускает `foreachPartition` со штатным числом партий.
4. На каждом исполнителе через `attachTransaction` создаётся локальный объект транзакции, в котором выполняется запись данных партии операцией `modifyRows`.
5. После завершения всех заданий записи драйвер выполняет `advanceConsumer` для каждого таблета очереди в той же транзакции.

6. Драйвер фиксирует транзакцию методом `commit`. При сбое любой задачи записи или последующих операций драйвер вызывает `abort`, и Spark переисполняет батч.
7. На драйвере метод `Source.commit` пропускает действия по продвижению смещения.

С точки зрения реализации решение 2 требует трёх компонентов, рассмотренных в §3.2: интерфейса `StreamingTransactionSupport`, регистрируемого через `ServiceLoader`, инструментации метода `MicroBatchExecution.runBatch` через Java-агент и пары `ThreadLocal`-контекстов для передачи идентификатора транзакции и параметров батча между методами драйвера. Дополнительно требуется расширение клиентской библиотеки YT функцией `attachTransaction`, что затрагивает соседний проект.

Сильная сторона решения — сохранение параллелизма записи. Каждый исполнитель обрабатывает свою партицию на своих ядрах, в своей JVM, и сериализация выходных строк, формирование запросов `modifyRows` и заполнение сетевого канала к грс-прокси выполняются параллельно на всех исполнителях. На пути к динамической таблице остаётся одно узкое место — общая грс-прокси, через которую проходят все запросы транзакции, — но оно является серверным компонентом и при равных ресурсах кратно превосходит пропускную способность одного исполнителя. Дополнительно, объём записи в рамках одной транзакции регулируется параметром `spark.yt.write.dynBatchSize`, который определяет количество строк, накапливаемых перед отправкой запроса `modifyRows`. Это позволяет балансировать между количеством запросов к грс-прокси и размером каждого запроса, адаптируясь под характеристики сети и производительность прокси.

Слабая сторона решения — повышенная сложность реализации и эксплуатации. Использование инструментации байт-кода Spark делает код чувствительным к смене минорных версий Spark и усложняет отладку: сбой в декорированном методе наблюдается как ошибка во внутреннем коде Spark, не имеющая прямого соответствия в исходных текстах SPYT. Расширение клиентской библиотеки YT функцией `attachTransaction` распространяет область изменений за пределы SPYT.

3.4.3 Сравнение решений

В таблице 3.1 приводится сравнение существующего частичного решения (§1.5), решения 1 и решения 2 по характеристикам, существенным для постановки задачи (§1.6).

Таблица 3.1 — Сравнение существующего и предлагаемых решений

Характеристика	Существующее решение (§1.5)	Решение 1	Решение 2
Целевой тип таблицы	сортированная динамическая	упорядоченная динамическая	упорядоченная динамическая
Семантика при произвольных преобразованиях	at-least-once	exactly-once	exactly-once
Семантика при stateless-преобразованиях с малым объёмом таблицы	exactly-once	exactly-once	exactly-once
Параллелизм записи	сохраняется	теряется	сохраняется
Узкие места записи	первичный ключ	один исполнитель и общая грс-прокси	общая грс-прокси

Характеристика	Существующее решение (§1.5)	Решение 1	Решение 2
Зависимость пропускной способности от объёма таблицы	деградация по мере роста	отсутствует	отсутствует
Зависимость пропускной способности от размера батча	линейная	ООМ-риск на больших батчах	линейная
Изменения в SPYT	нет	локальные в data-source-extended	в data-source-extended, spark-patch, spark-adapter
Изменения в сторонних проектах	нет	нет	расширение клиентской библиотеки YT
Использование инструментации байт-кода Spark	нет	нет	да

3.4.4 Обоснование выбора решения 2

Из таблицы 3.1 следует, что по постановке задачи (§1.6) ни существующее решение, ни решение 1 не удовлетворяют всем требованиям одновременно.

Существующее решение исключается двумя свойствами, явно противоречащими §1.6: оно ограничивает область применимости преобразованиями, не нарушающими однозначное соответствие между входной и выходной строкой, и его пропускная способность деградирует с ростом объёма целевой таблицы.

Решение 1 удовлетворяет требованиям корректности и применимости, но не удовлетворяет требованию по пропускной способности. По всем характеристикам производительности оно строго уступает решению 2: к ограничению со стороны общей грс-прокси, наследуемому обоими решениями, добавляется ограничение со стороны одного исполнителя, через который сериализуется запись всего батча. Преимущество решения 1 — простота реализации — окупается однократно, тогда как ограничение пропускной способности проявляется на каждом батче эксплуатации.

Решение 2 удовлетворяет всем требованиям §1.6: обеспечивает exactly-once при произвольных преобразованиях, не привязано к специфическому типу таблицы и сохраняет параллелизм записи, не уступая существующему решению по пропускной способности на задачах, где последнее применимо. Стоимость реализации, выраженная в инструментации байт-кода Spark и расширении клиентской библиотеки YT, является разовой и не влияет на эксплуатационные характеристики.

На основании изложенного к реализации принимается решение 2. Решение 1 в дальнейших главах работы не рассматривается, кроме контекстов, где оно явно полезно как точка сравнения.

3.5 Архитектура выбранного решения

В настоящем разделе фиксируется состав компонентов решения 2, распределение ответственности между ними, перечень изменений в существующих частях SPYT и поток управления при обработке одного микропакета. Формальное описание протокола атомарной фиксации, включая разбор поведения при сбоях, вынесено в §3.6.

3.5.1 Состав модулей и распределение ответственности

Реализация решения 2 затрагивает четыре модуля Gradle-проекта SPYT и одну точку расширения клиентской библиотеки YT. Распределение ответственности между ними соответствует разделению функциональности на устойчивые и изменяемые относительно версии Spark компоненты.

Модуль `data-source-extended` содержит существующие классы потокового источника `YtStreamingSource`, потокового приёмника `YtStreamingSink` и поддерживающие их `YtQueueOffset`, `YtQueueRange`, `YtQueueRDD`. В рамках решения 2 в этот модуль вносится альтернативная ветка обработки батча, активируемая флагом `spark.yt.streaming.transactional`, и реализация SPI `YTsaurusStreamingTransactionSupport`, инкапсулирующая жизненный цикл `tablet`-транзакции на драйвере.

Модуль `spark-adapter` содержит интерфейс `StreamingTransactionSupport` и пару `ThreadLocal`-контекстов, передающих идентификатор транзакции и параметры батча между фазами обработки на драйвере. Размещение интерфейса в `spark-adapter`, а не в `data-source-extended`, продиктовано необходимостью обращаться к нему из декоратора `MicroBatchExecution`, расположенного в `spark-patch`.

Модуль `spark-patch` содержит декоратор метода `MicroBatchExecution.runBatch`, помеченный аннотациями `@PatchSource` и `@Decorate` (см. §3.2.2). Декоратор оборачивает оригинальный метод вызовами начальной и завершающей фаз, делегируемыми реализации SPI, загруженной через `ServiceLoader`.

Модуль `yt-wrapper` содержит вспомогательные утилиты для работы с `tablet`-транзакциями `YTsaurus`. В нём вводится утилита, оборачивающая операцию `attachTransaction`, расширяющая клиентскую библиотеку YT.

Точкой расширения клиентской библиотеки YT является функция `attachTransaction(transactionId, proxyAddress)`, создающая локальный объект `ApiServiceTransaction`, направляющий запросы в указанный `grpc`-прокси с указанным идентификатором (см. §2.4.2, второй путь устранения изоляции).

3.5.2 Компонентная диаграмма

На рисунке 3.1 показан состав компонентов решения 2 и связи между ними. Прямоугольниками со скруглёнными углами обозначены процессы JVM (драйвер, исполнитель, `grpc`-прокси), сплошными прямоугольниками — модули и классы внутри процессов, стрелками — направление вызовов.

flowchart TB

```
subgraph Driver[JVM драйвера]
    MBE[MicroBatchExecution.runBatch]
    DEC[Декоратор runBatch<br/>spark-patch]
    SPI[YTsaurusStreamingTransactionSupport<br/>data-source-extended]
    SRC[YtStreamingSource]
    SINK[YtStreamingSink]
    TLC[YtStreamingTransactionContext<br/>YtStreamingBatchContext]
    QUO[YtQueueOffset.advance...]
end
subgraph Executor[JVM исполнителя]
    EXR[foreachPartition closure]
    ATT[attachTransaction<br/>yt-wrapper]
    MOD[modifyRows]
end
subgraph Proxy[grpc-прокси общего пула]
    TX[(tablet-транзакция)]
end
```

```

DEC --> MBE
DEC --> SPI
SPI --> TLC
MBE --> SRC
MBE --> SINK
SRC --> TLC
SINK --> TLC
SINK --> EXR
SPI --> QUO
SPI --> TX
QUO --> TX
EXR --> ATT
ATT --> MOD
MOD --> TX

```

Рисунок 3.1 — Компонентная диаграмма решения 2

Драйвер сосредотачивает управление жизненным циклом транзакции в реализации `SPI YtSaurusStreamingTransactionSupport`, вызываемой из декоратора `runBatch`. Передача идентификатора транзакции и параметров батча между методами драйвера происходит через `ThreadLocal`-контексты. Исполнители не открывают и не фиксируют транзакцию: они лишь присоединяются к ней через `attachTransaction` и направляют запросы `modifyRows` в тот же `grpc`-прокси, на котором она открыта.

3.5.3 Изменения в существующих компонентах SPYT

Перечень компонентов, затрагиваемых реализацией решения 2, и характер изменений приведён ниже.

1. `YtStreamingSource` — метод `getBatch` не изменяется; в методе `commit` при наличии активного контекста транзакции смещение потребителя продвигается в составе этой транзакции через `offsetProvider.advance(..., parentTransactionId)`; при отсутствии контекста и включённом флаге `spark.yt.streaming.transactional` метод завершается без действий (транзакция ещё не зафиксирована декоратором).
2. `YtStreamingSink` — в методе `addBatch` идентификатор открытой драйвером транзакции и адрес `grpc`-прокси извлекаются из `YtStreamingTransactionContext` и доставляются на исполнители широковещательными переменными; в замыкании `foreachPartition` создаётся локальный объект транзакции через `attachTransaction`, и запись `modifyRows` выполняется в этой транзакции вместо открытия собственной.
3. `YtDynamicTableWriter` — расширяется необязательным параметром `parentTransaction: Option[ApiServiceTransaction]`; при его наличии запросы `modifyRows` направляются в переданную транзакцию, а не открывают собственную.
4. `MicroBatchExecution.runBatch` (Apache Spark) — декорируется через механизм `spark-patch`. Декоратор перед вызовом оригинального метода открывает `tablet`-транзакцию и помещает её идентификатор и адрес прокси в `YtStreamingTransactionContext`; после возврата из оригинального метода вызывает `commitOffsets`, который через `Source.commit` продвигает смещение потребителя в составе той же транзакции, после чего фиксирует транзакцию; при исключении — отменяет транзакцию и откатывает запись в `commit`-лог Spark. Оригинальное поведение `runBatch` сохраняется без изменений.
5. Новый класс `YtSaurusStreamingTransactionSupport` (модуль `data-source-extended`) — реализация `SPI StreamingTransactionSupport`, регистрируемая в `META-INF/services/`. Инкапсулирует создание `tablet`-транзакции (`TransactionType.Tablet, sticky=true`), получение адреса `grpc`-прокси через рефлекссию, а также операции `commit` и `abort`.
6. Новый интерфейс `StreamingTransactionSupport` (модуль `spark-adapter`) — контракт жизненного цикла распределённой транзакции с методами `isTransactionalStreamingEnabled`, `createTransaction`, `setTransaction`,

- clearTransactionId, isRecoveryNeeded, markRecoveryNeeded, clearRecoveryNeeded.
- 7. Новый ThreadLocal-контекст YtStreamingTransactionContext (модуль data-source-extended) — хранит StreamingTxContext(txId, stickyAddress) и флаг recoveryNeededFlag.
- 8. Расширение yt-wrapper функцией attachTransaction(transactionId) — оборачивает операцию AttachTransaction клиентской библиотеки YT.

3.5.4 Поток управления при обработке батча

На рисунке 3.2 показан поток управления при штатной обработке одного микропакета в решении 2. Указаны только сообщения, относящиеся к управлению транзакцией; пользовательская логика и стандартные сообщения Spark опущены.

```
sequenceDiagram
    participant DEC as Декоратор runBatch
    participant SPI as YTsaurusStreamingTransactionSupport
    participant SRC as YtStreamingSource
    participant SINK as YtStreamingSink
    participant EX as Исполнитель
    participant PX as rpc-прокси
    DEC->>SPI: createTransaction
    SPI->>PX: startTransaction (TransactionType.Tablet)
    PX-->>SPI: transactionId, proxyAddress
    SPI->>SPI: setContext(txId, proxyAddress) in ThreadLocal
    DEC->>SRC: getBatch
    SRC-->>DEC: DataFrame
    DEC->>SINK: addBatch
    SINK->>SINK: read txId, proxyAddress from ThreadLocal
    SINK->>EX: foreachPartition (broadcast txId, proxyAddress)
    EX->>PX: attachTransaction(txId)
    EX->>PX: modifyRows on attached transaction
    EX-->>SINK: success
    DEC->>DEC: commitOffsets(availableOffsets)
    DEC->>SRC: commit(endOffset)
    SRC->>PX: advanceConsumer in transaction (txId)
    PX-->>SRC: ok
    SRC-->>DEC: ok
    DEC->>SPI: commit()
    SPI->>PX: commit transaction
    PX-->>SPI: ok
```

Рисунок 3.2 — Поток управления при штатной обработке батча в решении 2

Декоратор runBatch получает управление перед оригинальным методом и вызывает createTransaction SPI, которая открывает tablet-транзакцию (TransactionType.Tablet) через rpc-прокси общего пула и сохраняет её идентификатор и адрес прокси в YtStreamingTransactionContext. Оригинальный runBatch выполняется без изменений: вызывается Source.getBatch, формируется план, выполняется Sink.addBatch. На стороне приёмника идентификатор транзакции и адрес rpc-прокси извлекаются из YtStreamingTransactionContext и доставляются на исполнители широковебательными переменными; в замыкании foreachPartition создаётся локальный объект транзакции через attachTransaction, и запись modifyRows выполняется в нём. После возврата из оригинального runBatch декоратор вызывает commitOffsets, который обходит все источники и вызывает Source.commit(endOffset). В методе YtStreamingSource.commit при наличии активного контекста транзакции смещение потребителя продвигается в составе той же транзакции через advanceConsumer. Затем декоратор фиксирует транзакцию вызовом commit() на объекте StreamingTransactionHandle.

3.6 Протокол атомарной фиксации батча

TODO: Формализованное описание протокола атомарной фиксации батча в решении 2: формальная нотация шагов, инварианты после каждого шага, разбор поведения при сбое на каждом шаге (открытие транзакции, broadcast, modifyRows на исполнителе, потеря связи с грс-прокси, advanceConsumer на драйвере, commit), взаимодействие с offset/commit-логом Spark при штатной отмене и при переисполнении батча. Раздел дописывается после получения работающего прототипа решения 2 с целью уточнения деталей поведения, наблюдаемых в реализации.

3.7 Риски и компромиссы

В настоящем разделе перечислены риски, существенные для эксплуатации решения 2, и принятые архитектурные компромиссы. Под риском понимается событие или свойство среды, способное привести к деградации функциональных или нефункциональных требований §3.1; под компромиссом — осознанное архитектурное решение, ограничивающее одно из требований в обмен на удовлетворение другого.

3.7.1 Зависимость от внутреннего API Apache Spark

Декорирование метода `MicroBatchExecution.runBatch` через инструментацию байт-кода (§3.2.2) опирается на внутреннее, не специфицированное публично API Apache Spark. Изменение сигнатуры метода, его разбиение на несколько методов или существенное изменение последовательности вызовов внутри него в новых минорных версиях Spark способно сломать декоратор: либо в момент загрузки агента (несовпадение сигнатуры), либо в момент исполнения (несовпадение предположений о порядке вызовов `Source.getBatch` и `Sink.addBatch`).

Принятые меры смягчения. Декорируемые классы Apache Spark вынесены в модуль `spark-adapter`, имеющий по реализации на каждую поддерживаемую версию Spark, и подгружаются Java-агентом по результатам определения версии в момент инициализации (§3.2.2). Конкретный декоратор `runBatch` для решения 2 размещается в существующей инфраструктуре адаптеров и опирается на стабильные точки взаимодействия с `Source` и `Sink`, наблюдаемые во всех версиях Spark, поддерживаемых SPYT (3.2.x–3.5.x). Расширение поддерживаемого диапазона версий Spark в SPYT выполняется централизованно через `spark-adapter` и не требует отдельной работы по сопровождению декоратора решения 2.

3.7.2 Общая грс-прокси как точка отказа

Конфигурация с общим пулом грс-прокси (`spark.ytsaurus.grpc.job.proxy.enabled=false`) переносит точку отказа из `job-proxy` исполнителей в общий пул. Все запросы транзакции — `createTransaction`, `pingTransaction`, `modifyRows` с каждого исполнителя, `advanceConsumer`, `commit` — обслуживаются одним грс-прокси, на котором открыта транзакция. Отказ этого прокси в течение жизни транзакции приводит к её аварийному завершению независимо от состояния запросов записи на исполнителях.

Принятые меры смягчения. Во-первых, аварийное завершение транзакции не нарушает корректности: незафиксированная транзакция не оставляет следа ни в выходной таблице, ни в состоянии потребителя, и Spark переисполняет батч штатным образом. Во-вторых, общий пул грс-прокси `Ytsaurus` состоит из множества равноправных экземпляров с собственным резервированием на уровне инфраструктуры `Ytsaurus`, и среднее время между отказами одного прокси из пула существенно превосходит соответствующую величину для одиночного процесса исполнителя. В-третьих, на уровне SPYT не вводится дополнительной логики восстановления соединения: повторное открытие транзакции выполняется на следующем батче после переисполнения текущего.

3.7.3 Соотношение времени жизни tablet-транзакции и длительности батча

Tablet-транзакция YTsaurus имеет ограниченное время жизни, продлеваемое периодическими сообщениями pingTransaction (§2.3.3). Если обработка батча превышает таймаут транзакции и пинги не отправляются вовремя, прокси отменяет транзакцию автоматически, что наблюдается на драйвере как ошибка «No such transaction» при попытке commit.

Источниками превышения являются: длительные пользовательские преобразования внутри gunBatch, медленная сериализация выходных строк, замедленная сеть до прокси, длинные паузы сборки мусора на драйвере. Дополнительный риск — несоответствие между временем жизни транзакции, заданным при её открытии, и реальной длительностью батча на загруженных кластерах.

Принятые меры смягчения. Во-первых, пинг транзакции запускается в отдельном фоновом потоке драйвера сразу после открытия транзакции и останавливается при commit, abort или исключении в декораторе. Период пинга задаётся в долях от заявленного времени жизни транзакции. Во-вторых, время жизни транзакции конфигурируется параметром, доступным пользователю SPYT, и устанавливается с запасом относительно ожидаемой 99-й перцентиля длительности батча. В-третьих, при наблюдении регулярных таймаутов в эксплуатации увеличение времени жизни транзакции является штатной реакцией, не требующей изменения кода.

3.7.4 Разрыв соединения исполнителя с грс-прокси

Между моментами attachTransaction и завершением последнего modifyRows исполнитель удерживает соединение с грс-прокси, на котором открыта транзакция. Разрыв этого соединения наблюдается на исполнителе как сетевая ошибка при очередном modifyRows либо как ошибка ответа от прокси.

Принятые меры смягчения. Во-первых, исполнитель не пытается восстановить соединение или повторно присоединиться к транзакции: исключение, выброшенное из замыкания foreachPartition, передаётся в драйвер штатным механизмом Spark и обрабатывается как сбой задачи записи. Во-вторых, драйвер при первом сбое любой задачи записи отменяет транзакцию через abort и не пытается фиксировать частично завершённую запись. В-третьих, переисполнение батча выполняется штатным механизмом Spark Structured Streaming с тем же набором смещений источника, поскольку Source.commit не успел выполнить продвижение смещения. Корректность сохраняется в силу того, что отменённая tablet-транзакция не оставляет видимых строк в выходной таблице (§2.2.1).

3.7.5 Сводка компромиссов

Принятые архитектурные компромиссы решения 2 формулируются следующим образом.

- В обмен на сохранение параллелизма записи допускается зависимость от внутреннего API Apache Spark, разрешаемая инфраструктурой версионных адаптеров spark-adapter.
- В обмен на единственность узкого места допускается централизация запросов транзакции на общем грс-прокси, разрешаемая опорой на серверное резервирование YTsaurus.
- В обмен на простоту обработки сбоев допускается отсутствие частичного восстановления внутри батча: любой сбой приводит к отмене транзакции и переисполнению батча целиком, что согласуется с моделью Spark Structured Streaming.

4 РЕАЛИЗАЦИЯ

В настоящей главе описывается реализация решения 2, выбранного в §3.4.4. Изложение следует снизу вверх: сначала рассматриваются вспомогательные компоненты (контекст транзакции, расширение writer-a), затем модифицированные потоковые компоненты (источник, приёмник), и наконец — управляющий слой (декоратор `MicroBatchExecution`, SPI-интерфейс и его реализация).

4.1 Структура изменений в репозитории SPYT

Реализация затрагивает пять модулей Gradle-проекта SPYT.

data-source — базовый модуль записи в динамические таблицы. Изменение: класс `YtDynamicTableWriter` расширяется необязательным параметром родительской транзакции.

data-source-extended — модуль потоковой обработки. Изменения: модификация `YtStreamingSource.commit` и `YtStreamingSink.addBatch`; новые классы `YtStreamingTransactionContext` и `YtsaurusStreamingTransactionSupport`.

spark-adapter — модуль версионных адаптеров Spark. Изменение: новый интерфейс `StreamingTransactionSupport` и вспомогательный трейт `StreamingTransactionHandle` в модуле `api`.

spark-patch — модуль инструментации байт-кода. Изменение: новый декоратор `MicroBatchExecutionDecorators`, размещённый в `spark-adapter/impl/spark-3.2.2` и применяемый ко всем поддерживаемым версиям Spark через механизм `@Decorate`.

yt-wrapper — модуль утилит для работы с YTsaurus. Изменение: метод `attachTransaction` в трейте `YtTransactionUtils`, оборачивающий операцию `AttachTransaction` клиентской библиотеки YT.

4.2 Контекст транзакции

Передача идентификатора транзакции и адреса грс-прокси между методами драйвера реализована через `YtStreamingTransactionContext` — объект-синглтон, инкапсулирующий `InheritableThreadLocal`.

Хранимый тип — `case class StreamingTxContext(txId: String, stickyAddress: Option[String])`. Использование `InheritableThreadLocal` вместо `ThreadLocal` обеспечивает видимость контекста в дочерних потоках, порождаемых Spark при планировании задач.

Помимо контекста транзакции, объект хранит атомарный флаг `recoveryNeededFlag: AtomicBoolean`. Флаг устанавливается декоратором при сбое батча и сигнализирует о том, что состояние потребителя YTsaurus может расходиться с `commit`-логом Spark. При следующем батче декоратор логирует предупреждение и сбрасывает флаг после успешной фиксации.

Публичный API объекта:

```
object YtStreamingTransactionContext {
  def get: Option[StreamingTxContext]
  def setContext(ctx: StreamingTxContext): Unit
  def currentTransactionId: Option[String]
  def currentStickyProxyAddress: Option[String]
  def clearTransactionId(): Unit
  def isRecoveryNeeded: Boolean
  def markRecoveryNeeded(): Unit
  def clearRecoveryNeeded(): Unit
}
```

Жизненный цикл контекста в рамках одного батча: декоратор устанавливает контекст перед вызовом оригинального `runBatch` и очищает его в блоке `finally` после завершения батча — независимо от того, завершился ли батч успешно или с ошибкой.

4.3 Модификация потокового источника

4.3.1 Метод `getBatch`

Метод `YtStreamingSource.getBatch` не изменяется. Он формирует `YtQueueRDD` по диапазонам смещений и возвращает `DataFrame`. Смещение конца батча (`end: Offset`) передаётся Spark-ом в метод `commit` стандартным механизмом и не требует дополнительного сохранения в `ThreadLocal`.

4.3.2 Метод `commit`

Метод `YtStreamingSource.commit` реализует два режима работы в зависимости от наличия активного контекста транзакции.

Транзакционный режим (контекст установлен декоратором): смещение потребителя продвигается в составе уже открытой транзакции. Метод читает `txId` и `stickyAddress` из `YtStreamingTransactionContext`, при наличии `stickyAddress` создаёт клиент, зафиксированный на конкретном грс-прокси, и вызывает `offsetProvider.advance(..., parentTransactionId)`. Операция `advanceConsumer` выполняется в той же `tablet`-транзакции, что и запись данных на исполнителях.

Нетранзакционный режим с включённым флагом (контекст отсутствует, но `spark.yt.streaming.transactional=true`): метод завершается без действий. Это состояние возникает при сбое батча до установки контекста — транзакция не была открыта, и продвигать смещение не нужно.

Нетранзакционный режим (флаг выключен): штатное поведение — `offsetProvider.advance(..., None)` без транзакции.

```
override def commit(end: Offset): Unit = {
  val txContext = YtStreamingTransactionContext.get

  if (txContext.isEmpty) {
    val transactionalEnabled =
      sqlContext.sparkSession.ytConf(Streaming.Transactionnal)
    if (transactionalEnabled) {
      return
    }
    offsetProvider.advance(consumerPath, YtQueueOffset(end),
      lastCommittedOffset, maxOffset, None)(yt)
    return
  }

  val parentTransactionId = txContext.map(_.txId)
  val stickyAddr = txContext.flatMap(_.stickyAddress)
  val advanceClient: CompoundClient = stickyAddr match {
    case Some(addr) =>
      val conf =
        YtClientConfigurationConverter.ytClientConfiguration(...)
        YtClientProvider.ytClientFixedProxy(conf, addr)
    case None => yt
  }
}
```



```

offsetProvider.advance(consumerPath, YtQueueOffset(end),
    lastCommittedOffset, maxOffset,
    parentTransactionId)(advanceClient)
}

```

4.4 Модификация потокового приёмника

Метод `YtStreamingSink.addBatch` расширен поддержкой транзакционного режима. Изменения сосредоточены в двух местах: подготовка широковещательных переменных и создание клиента `YTsaurus` на исполнителе.

На драйвере метод читает `YtStreamingTransactionContext.get` и извлекает `txId` и `stickyAddress`. Оба значения упаковываются в широковещательные переменные `bcParentTransactionId` и `bcStickyProxyAddress`:

```

val txContext = YtStreamingTransactionContext.get
val bcParentTransactionId = txContext.map(ctx =>
    sparkContext.broadcast(ctx.txId))
val bcStickyProxyAddress =
    txContext.flatMap(_.stickyAddress).map(sparkContext.broadcast)

```

На исполнителе в замыкании `foreachPartition` выбор клиента `YTsaurus` зависит от наличия `stickyAddress`. При его наличии создаётся клиент, зафиксированный на конкретном грс-прокси (`YtClientProvider.ytClientFixedProxy`), что гарантирует направление запросов `attachTransaction` и `modifyRows` на тот же прокси, где открыта транзакция. При отсутствии — используется клиент из общего пула:

```

implicit val partitionYtClient: CompoundClient = stickyAddr match {
  case Some(addr) =>
    YtClientProvider.ytClientFixedProxy(bcYtClientConfiguration.value,
    addr)
  case None =>
    YtClientProvider.ytClient(bcYtClientConfiguration.value)
}
val attachedTx = txId.map(id => YtWrapper.attachTransaction(id))
val dynamicTableWriter = new YtDynamicTableWriter(bcPath.value,
    bcSchema.value,
    bcWriterConfig.value, bcParameters.value, attachedTx)

```

Объект `attachedTx: Option[ApiServiceTransaction]` передаётся в конструктор `YtDynamicTableWriter`. При его наличии все запросы `modifyRows` направляются в присоединённую транзакцию; при отсутствии — поведение не изменяется относительно нетранзакционного режима.

4.5 Расширение `YtDynamicTableWriter`

Класс `YtDynamicTableWriter` расширен необязательным параметром `parentTransaction: Option[ApiServiceTransaction] = None`:

```

class YtDynamicTableWriter(richPath: YPathEnriched, schema:
    StructType,
    wConfig: SparkYtWriteConfiguration, options: Map[String, String],
    parentTransaction: Option[ApiServiceTransaction] = None)
    (implicit ytClient: CompoundClient) extends OutputWriter

```

Параметр имеет значение по умолчанию `None`, что обеспечивает полную обратную совместимость: все существующие вызовы конструктора без явного указания транзакции продолжают работать без изменений.

Внутри класса параметр используется в методе `commitBatch`:

```
private def commitBatch(): Unit = {
    val request = modifyRowsRequestBuilder.build()
    YtWrapper.insertRows(request, parentTransaction)
    initBatch()
}
```

Метод `YtWrapper.insertRows` принимает `Option[ApiServiceTransaction]`: при `Some(tx)` запрос `ModifyRows` направляется через объект транзакции (`tx.modifyRows(request)`), при `None` — через обычный клиент `YTsaurus`.

Батчирование записи сохраняется: строки накапливаются в `ModifyRowsRequest.Builder` до достижения `wConfig.dynBatchSize`, после чего выполняется `commitBatch`. Параметр `dynBatchSize` управляется конфигурацией `spark.yt.write.dynBatchSize` и позволяет регулировать объём одного запроса `modifyRows` в рамках транзакции.

С точки зрения масштабируемости, батчирование играет ключевую роль. Поскольку все исполнители направляют запросы `modifyRows` в одну общую грс-прокси, количество запросов к прокси прямо влияет на нагрузку на неё. Слишком малое значение `dynBatchSize` приводит к большому числу мелких запросов, увеличивая накладные расходы на сетевое взаимодействие и обработку на прокси. Слишком большое значение увеличивает размер каждого запроса, что может привести к росту задержек и потреблению памяти на исполнителях. Оптимальное значение зависит от характеристик сети, производительности грс-прокси и объёма данных в батче, и подбирается экспериментально для конкретной конфигурации кластера.

4.6 Декоратор `MicroBatchExecution`

Декоратор `MicroBatchExecutionDecorators` подменяет метод `runBatch` класса `MicroBatchExecution` через механизм инструментации байт-кода `spark-patch`. Аннотации `@Decorate` и `@OriginClass` указывают агенту целевой класс и метод для подмены; оригинальный метод остаётся доступным под именем `__runBatch`.

Логика декоратора:

```
@DecoratedMethod
private def runBatch(sparkSessionToRunBatch: SparkSession): Unit = {
    val sts = StreamingTransactionSupport.instance
    val transactionalStreamingEnabled =
        sts.isTransactionalStreamingEnabled(sparkSessionToRunBatch)

    if (!transactionalStreamingEnabled) {
        __runBatch(sparkSessionToRunBatch)
        return
    }

    if (sts.isRecoveryNeeded) {
        logWarning("...")
    }

    val currentTransaction: StreamingTransactionHandle =
        sts.createTransaction(sparkSessionToRunBatch)
    sts.setTransaction(currentTransaction)
    val batchIdAtEntry = currentBatchId
    var commitLogWritten = false

    try {
        __runBatch(sparkSessionToRunBatch)
        commitLogWritten = true
        MicroBatchExecutionDecorators.commitOffsets(availableOffsets)
    }
```

```

        currentTransaction.commit()
        sts.clearRecoveryNeeded()
    } catch {
        case e: Exception =>
            sts.markRecoveryNeeded()
            try { currentTransaction.abort() } catch { case _: Throwable =>
                () }
            if (commitLogWritten) {
                MicroBatchExecutionDecorators.deleteCommitLogEntry(commitLog,
                    batchIdAtEntry)
            }
            throw e
    } finally {
        sts.clearTransactionId()
    }
}

```

При выключенном флаге декоратор прозрачно делегирует вызов оригинальному методу и не вносит накладных расходов.

При включённом флаге последовательность действий следующая.

1. Через SPI создаётся tablet-транзакция; её идентификатор и адрес грс-прокси сохраняются в `YtStreamingTransactionContext`.
2. Вызывается оригинальный `__runBatch`, внутри которого Spark вызывает `Source.getBatch`, планирует и исполняет задания записи через `Sink.addBatch`.
3. После успешного возврата из `__runBatch` декоратор вызывает `commitOffsets(availableOffsets)`, который обходит все источники и вызывает `source.commit(offset)`. В `YtStreamingSource.commit` смещение потребителя продвигается в составе открытой транзакции.
4. Транзакция фиксируется вызовом `currentTransaction.commit()`.
5. Флаг `recoveryNeeded` сбрасывается.

При исключении на любом из шагов 2–4:

- устанавливается флаг `recoveryNeeded`;
- транзакция отменяется (`abort`);
- если исключение произошло после записи в `commit`-лог Spark (`commitLogWritten = true`), запись откатывается через `commitLog.purgeAfter(batchId - 1)`, чтобы Spark переисполнил батч с теми же смещениями.

Вспомогательный метод `commitOffsets` вынесен в объект-компаньон и вызывает `source.commit` только для источников типа `Source (V1 API)`, что соответствует реализации `YtStreamingSource`.

4.7 SPI-точка расширения

4.7.1 Интерфейс `StreamingTransactionSupport`

Интерфейс `StreamingTransactionSupport` объявляет контракт жизненного цикла распределённой транзакции:

```

trait StreamingTransactionSupport {
    def isTransactionalStreamingEnabled(sparkSession: SparkSession):
        Boolean
    def createTransaction(sparkSession: SparkSession):
        StreamingTransactionHandle
    def setTransaction(handle: StreamingTransactionHandle): Unit
    def setTransactionId(txId: String): Unit
    def setStickyProxyAddress(address: Option[String]): Unit
}

```

```

def clearTransactionId(): Unit
def isRecoveryNeeded: Boolean
def markRecoveryNeeded(): Unit
def clearRecoveryNeeded(): Unit
}

```

Вспомогательный трейт `StreamingTransactionHandle` представляет открытую транзакцию:

```

trait StreamingTransactionHandle {
  def getId: String
  def getStickyProxyAddress: Option[String]
  def commit(): Unit
  def abort(): Unit
}

```

Загрузка реализации выполняется через `ServiceLoader` при первом обращении:

```

object StreamingTransactionSupport {
  lazy val instance: StreamingTransactionSupport =
    ServiceLoader.load(classOf[StreamingTransactionSupport]).findFirst().get()
}

```

Размещение интерфейса в модуле `spark-adapter/api` (а не в `data-source-extended`) обусловлено тем, что декоратор `MicroBatchExecutionDecorators` в `spark-patch` должен обращаться к нему без циклической зависимости.

4.7.2 Реализация `YTsaurusStreamingTransactionSupport`

Класс `YTsaurusStreamingTransactionSupport` регистрируется в `META-INF/services/tech.ytsaurus.spyt.adapter.StreamingTransactionSupport`.

Метод `createTransaction` создаёт `tablet`-транзакцию с параметром `sticky=true`, что соответствует `TransactionType.Tablet` в клиентской библиотеке YT:

```

override def createTransaction(sparkSession: SparkSession):
  StreamingTransactionHandle = {
    val ytClient = YtClientProvider.ytClient(
      YtClientConfigurationConverter.ytClientConfiguration(sparkSession.sparkContext.getCo
    )
    val transaction = YtWrapper.createTransaction(
      parent = None,
      timeout = Duration.ofMinutes(5),
      sticky = true
    )(ytClient)
    new YtStreamingTransactionHandle(transaction)
  }
}

```

Класс `YtStreamingTransactionHandle` оборачивает `ApiServiceTransaction`. Адрес грс-прокси, на которой открыта транзакция, получается через рефлексия — метод `getRpcProxyAddress` не входит в публичный API `ApiServiceTransaction`, поэтому вызывается через `getDeclaredMethods` с `setAccessible(true)`. При недоступности метода (например, в будущих версиях клиентской библиотеки) возвращается `None`, и исполнители используют клиент из общего пула без фиксации на конкретном прокси.

Методы `commit` и `abort` делегируют вызовы объекту `ApiServiceTransaction` с блокирующим ожиданием (`join()`). В `abort` перехватывается `IllegalStateException`, возникающий при попытке отменить уже завершённую транзакцию.

4.8 Конфигурация и обратная совместимость

Транзакционный режим управляется конфигурационным параметром `spark.yt.streaming.transactional` (тип `Boolean`, значение по умолчанию `false`). Параметр объявлен в объекте `SparkYtConfiguration.Streaming`:

```
object Streaming {  
  case object Transactional extends ConfigEntry[Boolean]  
    ("streaming.transactional", Some(false))  
}
```

При значении `false` (по умолчанию) поведение всех компонентов идентично поведению до введения транзакционного режима:

- декоратор `MicroBatchExecutionDecorators` прозрачно делегирует вызов оригинальному `runBatch` без создания транзакции;
- `YtStreamingSource.commit` выполняет штатное продвижение смещения без транзакции;
- `YtStreamingSink.addBatch` не читает контекст транзакции; `bcParentTransactionId` и `bcStickyProxyAddress` равны `None`; `YtDynamicTableWriter` создаётся с `parentTransaction = None`.

Таким образом, существующие потоковые задания, не устанавливающие параметр `spark.yt.streaming.transactional=true`, продолжают работать без изменений и без каких-либо накладных расходов, связанных с транзакционным режимом.

Для включения транзакционного режима дополнительно требуется конфигурация `spark.ytsaurus.grpc.job.proxy.enabled=false`, переключающая исполнители на общий пул grpc-прокси кластера. Без этой настройки исполнители используют локальные `job-proxy`, изолированные от прокси драйвера, и операция `attachTransaction` завершится ошибкой «No such transaction» (см. §2.4.2).

5 ТЕСТИРОВАНИЕ

TODO: Глава будет написана после прогона тестов и сбора метрик для обоих архитектурных вариантов. Ниже приведён скелет подразделов.

5.1 Стратегия тестирования

TODO: Описание уровней тестирования: модульное, интеграционное, end-to-end. Обоснование выбора стратегии.

5.2 Тестовая инфраструктура

TODO: Описание окружения: локальный `YTsaurus` в `Docker`, `mini-Spark` в тестах, перечень существующих тестовых классов (`YTsaurusStreamingTest`, `YtStreamingSourceTest`, `YtTransactionalStreamingTest`).

5.3 Модульные тесты

TODO: Описание модульных тестов вспомогательных классов (`YtQueueUtilsTest`, тесты `YtStreamingBatchContext`).

5.4 Интеграционные тесты

5.4.1 Сценарий «успешный батч»

TODO: Описание сценария и проверки exactly-once семантики на штатном пути.

5.4.2 Сценарий «сбой между записью и продвижением смещения»

TODO: Описание моделирования сбоя и ожидаемого поведения: ноль строк в выходной таблице, смещение потребителя не продвинуто.

5.4.3 Сценарий «сбой после фиксации транзакции»

TODO: Описание моделирования сбоя на этапе записи Spark в журнал коммитов и проверки идемпотентности при повторном чтении.

5.5 Проверка строго однократной обработки

TODO: Методика подсчёта дубликатов, результаты прогонов для обоих вариантов реализации.

5.6 Сравнение архитектурных вариантов

TODO: Сравнение вариантов А и В по ключевым метрикам: пропускная способность, задержка батча, устойчивость к сбоям, потребление ресурсов.

5.7 Обоснование итогового выбора

TODO: Зафиксированный выбор варианта реализации с обоснованием на основе результатов раздела 5.6.

5.8 Известные ограничения

TODO: Перечень ограничений выбранного решения, не покрытых тестами сценариев и направлений развития.

ЗАКЛЮЧЕНИЕ

TODO: Заключение будет написано после завершения экспериментальной части и фиксации выбранного варианта реализации. Структура заключения:

1. Итоги по поставленным задачам в соответствии с введением.
2. Перечень полученных результатов: реализованный модуль, тесты, метрики производительности.
3. Практическая значимость работы.
4. Ограничения текущего решения.
5. Направления дальнейшей работы.